



Dorian Janiak

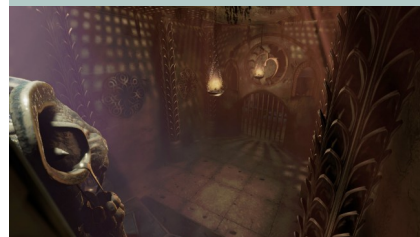
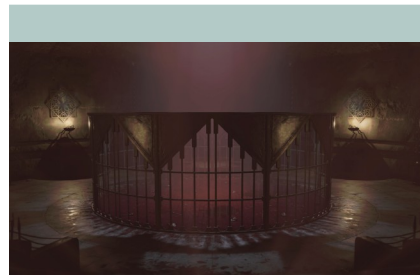
DORIANJANIAK.GITHUB.IO



Graphical remake of a single chamber from the classic game Legacy of Kain: Soul Reaver. While based on the original level design, the environment was recreated entirely from scratch and enhanced with additional detail, visual effects, and selective redesigns to create a cohesive experience while keeping distinctive to the original atmosphere. From technical side my primary goal was to learn the PBR (Physically Based Rendering) material workflow.

Working in Blender, I went through the entire asset creation pipeline for the first time. I practiced sculpting, retopology, and baking PBR texture maps (diffuse, normal, roughness, AO, height). I also experimented with optimization techniques like creating LODs (Levels of Detail), using trim sheets, and preparing geometry for layered materials and cloth simulation in Unity 3D.

Inside Unity 3D, I focused on assembling the scene and making it interactive. I wrote C# scripts to handle basic movement and to trigger object state machines, set up the layered materials, and configured the audio and graphics settings. Additionally, I gained hands-on experience with Unity's prefab management, lighting layers, volumetric effects, and built simple particle effects for environmental fire.



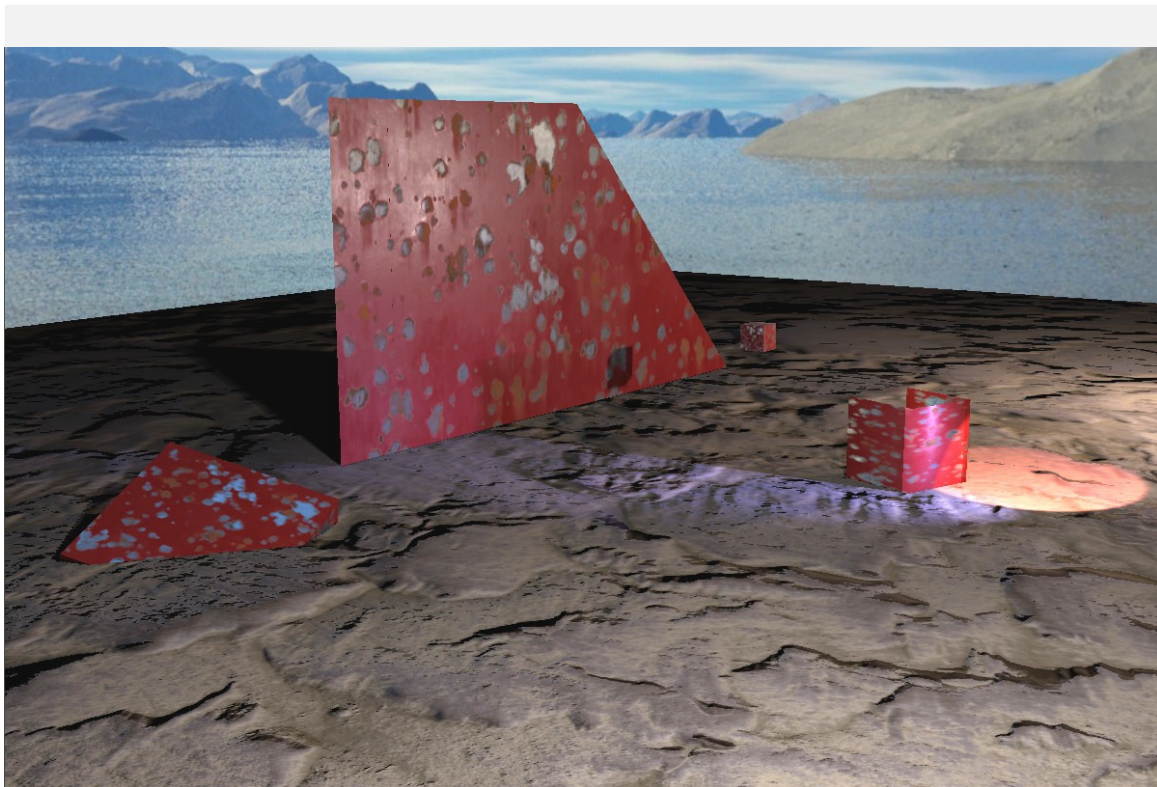
This was a hands-on learning project with the primary goal of getting familiar with Unreal Engine 5's core tools and its workflow. Instead of building a complete game, I created small 3D environment, focusing on understanding how different engine features work together in practice.

The scene was built mainly through kit-bashing using Quixel Megascans, which allowed me to test Nanite and see how the engine handles high-poly assets. On the technical side, I experimented with creating landscapes using terrain tools, setting up basic multi-texturing, and working with material instancing, which is a bit different in UE5 than in Unity 3D.

I also tested UE5's visual effects by implementing localized volumetric fog and simple Niagara particle systems. For assets that weren't available in the Megascans library, I modeled a few basic pieces myself in Blender.



```
public class SystemManager  
for ( async func  
if (status =  
constin
```



Starting in 2025, I have been developing a custom graphics engine from scratch. This project serves as a personal sandbox for experimentation and growth in two main areas: mastering the OpenGL graphics pipeline and writing efficient code using modern C++ features (C++ 11, 14, and 17). Rather than aiming to compete with commercial engines, my goal is to explore low-level graphics, high-performance optimization and software architecture. It is not intended to become fully featured product. It is a long-term research and learning project that I will repeatedly rebuild and improve in order to evaluate the impact of various implementation approaches on rasterization performance.

The project is currently in the early stages of intensive development and architectural refactoring, as I am actively redesigning the core structure to improve on initial design choices.

Future development of the engine will focus on expanding its capabilities with features such as cascaded shadow maps, depth of field, reflection probes, and volumetric effects. More importantly, improvements to the core architecture will include object visibility culling, render queue sorting, render passes, and efficient framebuffer management. In the longer term, I plan to extend the engine with multithreaded components to improve scalability and performance.